RFR27 16-Channel Temperature Monitor New Member Project-10

Rutgers Formula Racing

Maaz Mansuri, Joshua Scallo, Fayella Fenton, Gavin Palacios
October 20, 2026



Contents

1 Revision Log **2**

2 Purpose **2**

3 Parties Involved **2**

4 Hardware Design **3**

5 Firmware Design **4**
5.1 Introduction and Firmware Overview 4
5.2 Pin Mapping and Peripheral Allocation 6
5.3 Overview of functions 6
5.3.1 Channel Addressing: `setMuxChannel` 6
5.3.2 Signal Acquisition: `readChannelVoltage` 7
5.3.3 Mapping our voltage readings into temperature values: `voltagetoTemperature` 7
5.3.4 I2C Implementation: `requestData` 7
5.4 Architectural Decisions and Design Trade-offs 7
5.4.1 Settling Delay Choice ($20\,\mu$s) 7
5.4.2 Data Typing and Precision (`float` vs. `int`) 8

6 Verification, Calibration, & Test Results **8**

7 Bill of Materials **8**

8 Sources **9**

1 Revision Log

Editor	Date	Revision
Maaz Mansuri	Oct. 12 2026	Document Creation
Maaz Mansuri	Oct. 15 2026	Established outline of documentation and table of contents
Fayella Fenton	Oct. 15 2026	Inserted Source Refs
Joshua Scallo	Oct. 17 2026	Added schematic and hardware descriptions
Maaz Mansuri	Oct. 19 2026	Completed firmware Design and added flowcharts for the circuit and the code

2 Purpose

Create a 16-channel temperature monitor for the accumulator container and transmit pack thermal data over the I2C bus upon master query.

3 Parties Involved

Name	Team Role	Report Role
Maaz Mansuri	Software	Author
Joshua Scallo	Hardware	Author
Fayella Fenton	Hardware	Author
Vaanika Singla	Electronics Lead	Reviewer
Jeevan Shah	Electronics Lead	Reviewer

4 Hardware Design

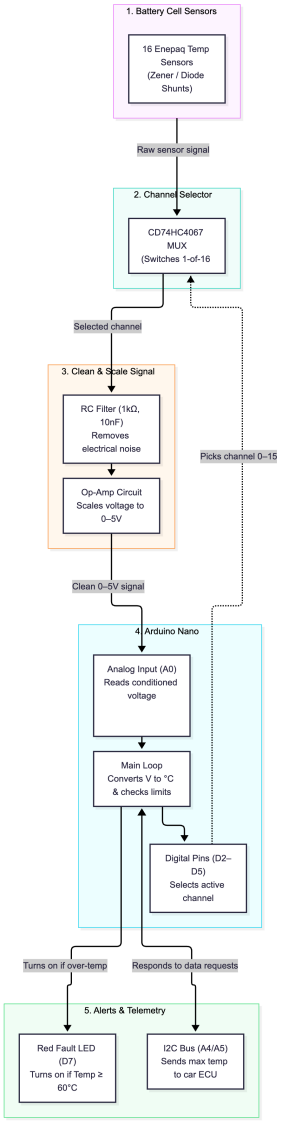


Figure 1: Hardware System Signal-Flow and Architecture Diagram

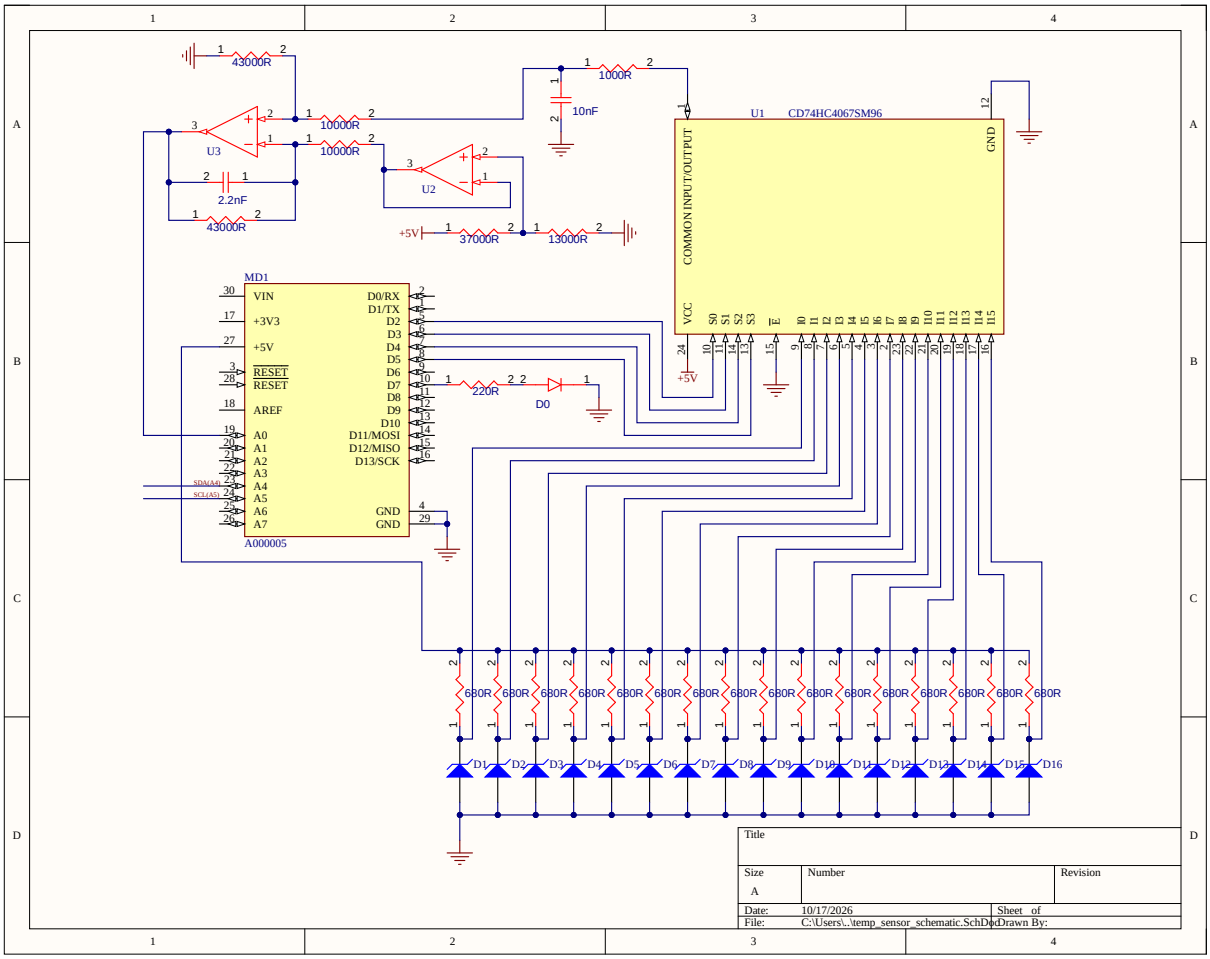


Figure 2: Schematic

Our schematic contains 16 temperature sensors represented by zener diodes, a 16-channel multiplexer (CD74HC4067SM96), signal conditioning circuitry, and an Arduino Nano. This design allows the use of the Arduino Nano's onboard ADC to collect the voltage readings from each temperature sensor one by one with help from the 16-channel multiplexer. This is achieved by connecting each channel of the multiplexer to a temperature sensor reading. These channels are then controlled by the firmware in the Arduino Nano's D2-D5 pins, where each chosen output of the multiplexer is first conditioned, then sent to the A0 pin of the Arduino Nano, where the voltage is gathered and tested through the firmware. The LED connected to the D7 pin then shines when the voltage reading scans as a temperature reading higher than 60 C.

5 Firmware Design

5.1 Introduction and Firmware Overview

The software is based on our planned wiring implementation and serves four main functions per the project instructions: reading each sensor's analog signal, converting it to a voltage, determining the highest temperature among the sensors, and reporting that value over I2C. While the original schematic includes a DC-DC converter and a 16:1 MUX, the converter is omitted here and an 8:1 MUX is used instead. This reduces the select pin count from four to three, while keeping the core firmware logic the same.

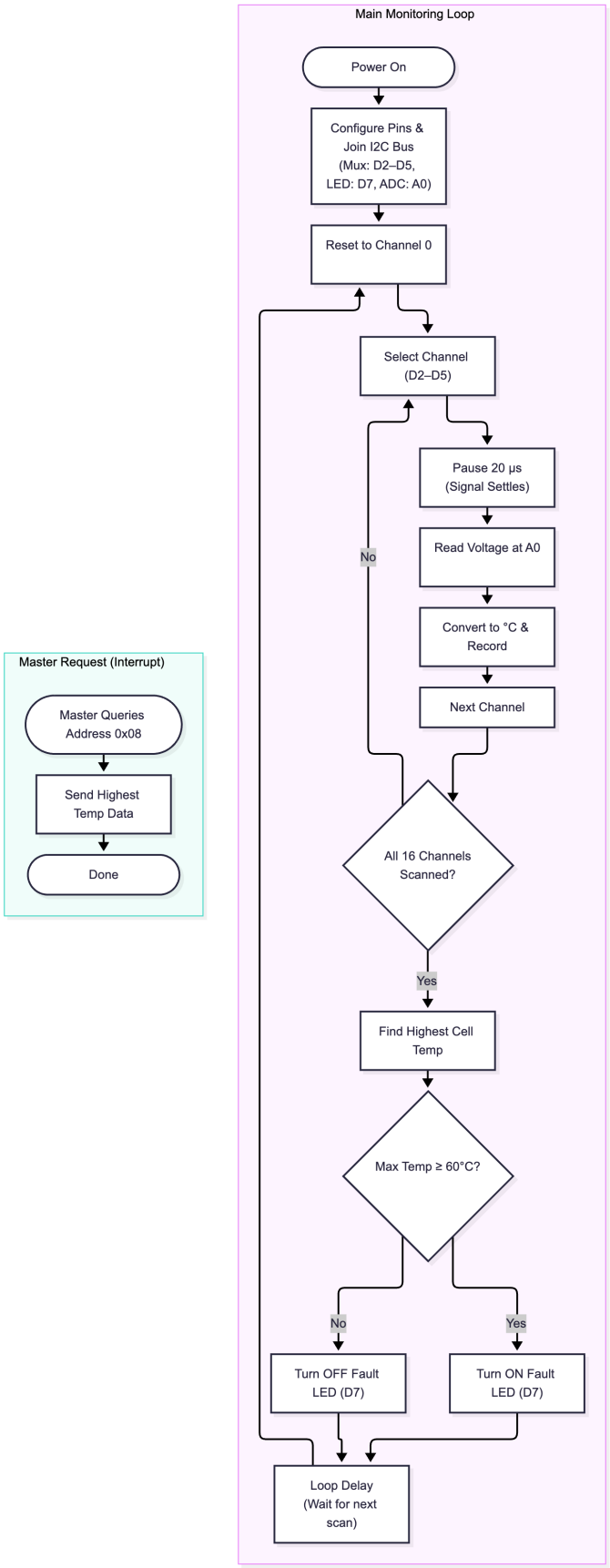


Figure 3: Software Flowchart

The core execution loop performs the following operations:

1. Iterates sequentially through channels 0 to 7 using the binary addresses from the select pins
2. Pauses for an explicit interval ($20\,\mu$s) to discharge/charge front-end capacitance before sampling.
3. Samples the conditioned analog node via `analogRead(A0)` and converts the raw 10-bit count into an engineering voltage ($0.0\,\text{V} - 5.0\,\text{V}$).
4. Maps the measured voltage to temperature (°C) based on the Enepaq sensor's temperature and voltage mapping datasheet.
5. Evaluates cell temperatures against 60°C critical limit, asserting the fault LED on pin `D7` if overheating is detected.
6. Stores the maximum pack temperature to prepare for for I2C request

5.2 Pin Mapping and Peripheral Allocation

To match the schematic in Figure 2, the hardware-to-software pin definitions are established as constants prior to compilation:

Table 1: Microcontroller Pin Allocation

Constant Name	Nano Pin	Direction	Hardware Functional Description
`muxSIG`	A0	Input	Conditioned analog voltage from multiplexer op-amp stage
`muxS0`	D2	Output	Binary address select line: Bit 0 (2^0)
`muxS1`	D3	Output	Binary address select line: Bit 1 (2^1)
`muxS2`	D4	Output	Binary address select line: Bit 2 (2^2)
`muxS3`	D5	Output	Binary address select line: Bit 3 (2^3)
`faultLED`	D7	Output	Over-temperature visual indicator LED
`SDA`	A4	Bidirectional	I2C Data line (configured automatically via `Wire.h`)
`SCL`	A5	Input	I2C Clock line (configured automatically via `Wire.h`)

```
#include <Wire.h>
//Defiing all of our pins here

Signal Pin from multiplexer to the Arduino
const int muxSIG = A0;

Pins that will select what channel to read in binary
const int muxS0 = 2;
const int muxS1 = 3;
const int muxS2 = 4;


//Defining address for Arduino as Slave
const byte SLAVE_ADDRESS = 0x08;
```

Listing 1: Pin definitions

5.3 Overview of functions

5.3.1 Channel Addressing: `setMuxChannel`

The CD74HC4067 requires a 4-bit binary code on pins `S0`–`S3` to connect one of the 16 analog input channels to the common output pin. Instead of manually iterating through binary, we call this function to input the desired channel we want to read, and then it will convert it to the binary equivalent using Arduino's bit read function.

```
void setMuxChannel(byte channel)  {
  //Allows you to input an integer and then it converts it into binary for the signal pins to read and select the
     channel
    digitalWrite(muxS0, bitRead(channel, 0));
    digitalWrite(muxS1, bitRead(channel, 1));
    digitalWrite(muxS2, bitRead(channel, 2));
}
```

Listing 2: setMuxChannel Function

5.3.2 Signal Acquisition: `readChannelVoltage`

This function switches the channel, pauses for the signal to stabilize, takes a 10-bit analog sample, and converts the integer output to a floating-point voltage:

$$V_{\text{measured}} = \text{ADC} \times \left(\frac{5.0\,\text{V}}{1023.0} \right) \tag{1}$$

```
float readChannelVoltage(byte channel) {
  setMuxChannel(channel);
  delayMicroseconds(20);

  int rawADC = analogRead(muxSIG);
  return (rawADC * 5.0f) / 1023.0f; //to scale the value and actually put it within our 0-5 range

}
```

Listing 3: readChannelVoltage Function

5.3.3 Mapping our voltage readings into temperature values: `voltagetoTemperature`

The Enepaq sensor have a table in the data sheet that map out the voltges readings to actually temperature with a certain range. The formula is yet to be determined, but once implemented the function will be able to take in the float voltage value and return a temperature value.

```
float voltagetoTemperature(float voltage) {
    //Find the reference values and the formula to convert it
    return 0.0f
}
```

Listing 4: Voltage to Temperature Conversion

5.3.4 I2C Implementation: `requestData`

When the bus master addresses the slave node 0x08, the hardware triggers the request data function.

```
void requestData() {
    // Send highest recorded pack temperature as a single byte
    Wire.write(maxTempByte);
}
```

Listing 5: I2C Slave Transmission Callback

5.4 Architectural Decisions and Design Trade-offs

5.4.1 Settling Delay Choice ($20\,\mu$s)

- **Context:** Inside the multiplexer, each switch adds about $70\,\Omega$ to $180\,\Omega$ of internal resistance (R_{ON}). This sits right before our input filter stage, which uses a $1\,\text{k}\Omega$ resistor and a $10\,\text{n}$F capacitor before feeding the signal to the op-amp.

- **Alternative Considered:** Reading the ADC right after changing the channel address pins, with zero delay in between.
- **Decision & Rationale:** If we sample immediately, voltage left over on the capacitor from the previous channel will bleed into the new reading. For example. The capacitor needs a moment to charge up to the new voltage level. We can find how long this takes using the circuit's RC time constant:

$$\tau = R_{\text{total}} \times C \approx (1000\,\Omega + 180\,\Omega) \times 10\,\text{nF} = 11.8\,\mu\text{s} \tag{2}$$

  Waiting $20\,\mu$s gives the signal about 1.7τ to settle down, ensuring our ADC reading reflects the true sensor voltage rather than residual charge from the channel before it. Across all 8 channels, this delay is negligible and we would not hinder our target of 1000hz sampling rate.

5.4.2 Data Typing and Precision (`float` vs. `int`)

- **Context:** Choosing the internal representation for voltage readings and temperature calculations.
- **Decision & Rationale:** With a sensitivity of $-9.5\,\text{mV}/^\circ\text{C}$, a 1°C temperature delta corresponds to a minute voltage change (≈ 1.94 ADC counts). Using integer arithmetic would cause severe truncation errors, masking gradual thermal build-up. Single-precision 32-bit floating-point numbers (`float`) are used for conversion math to preserve sub-degree precision for over-temperature detection.

6 Verification, Calibration, & Test Results

7 Bill of Materials

8 Sources

References

[] Understanding Our Components

[1] Robotics Back-End, *Arduino Uno - Pins Overview*, YouTube, 2021. Available: https://youtu.be/bniUECtJkeU

[2] Electronoobs, *Arduino Multiplexor - MUX INPUTS & OUTPUTS - ANALOG and PWM*, YouTube, 2021. Available: https://youtu.be/Dco6jo9xgAo

[3] Dejan, *How I2C Communication Works?*, HowToMechatronics. Available: https://howtomechatronics.com/tutorials/arduino/how-i2c-communication-works-and-how-to-use-it-with-arduino/

[4] CircuitBread, *Intro to Op-Amps (Operational Amplifiers) | Basic Circuits*, YouTube, 2022. Available: https://youtu.be/fmRHDqcodS4

[5] Temperature Sensor Datasheet. Available: https://enepaq.com/wp-content/uploads/2025/02/Sony-Murata-VTC5A-Li-ion-Battery-Module-With-Temperature-Sensor-Datasheet-.pdf

[] Circuit Basics

[6] RimstarOrg, *How to Read a Schematic*, YouTube, 2014. Available: https://youtu.be/_HZ-EQ8Hc8E

[7] EEVblog, *Playlist on DC Fundamentals*, YouTube, 2017. Available: https://www.youtube.com/show/VLPLZJkSyc12jDY?sbp=KgtZZGJobm1BNE05Z0AB

[] Arduino IDE